

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522 FİNAL PROJE RAPORU

Enes Canbay

22290024

Videolar:

https://drive.google.com/drive/folders/19KuXX8eWMeUCNJBS68ChYqFzfxUaG05g?usp=drive_link

Github:

<https://github.com/enescnby/BLM4522>

Haziran, 2026

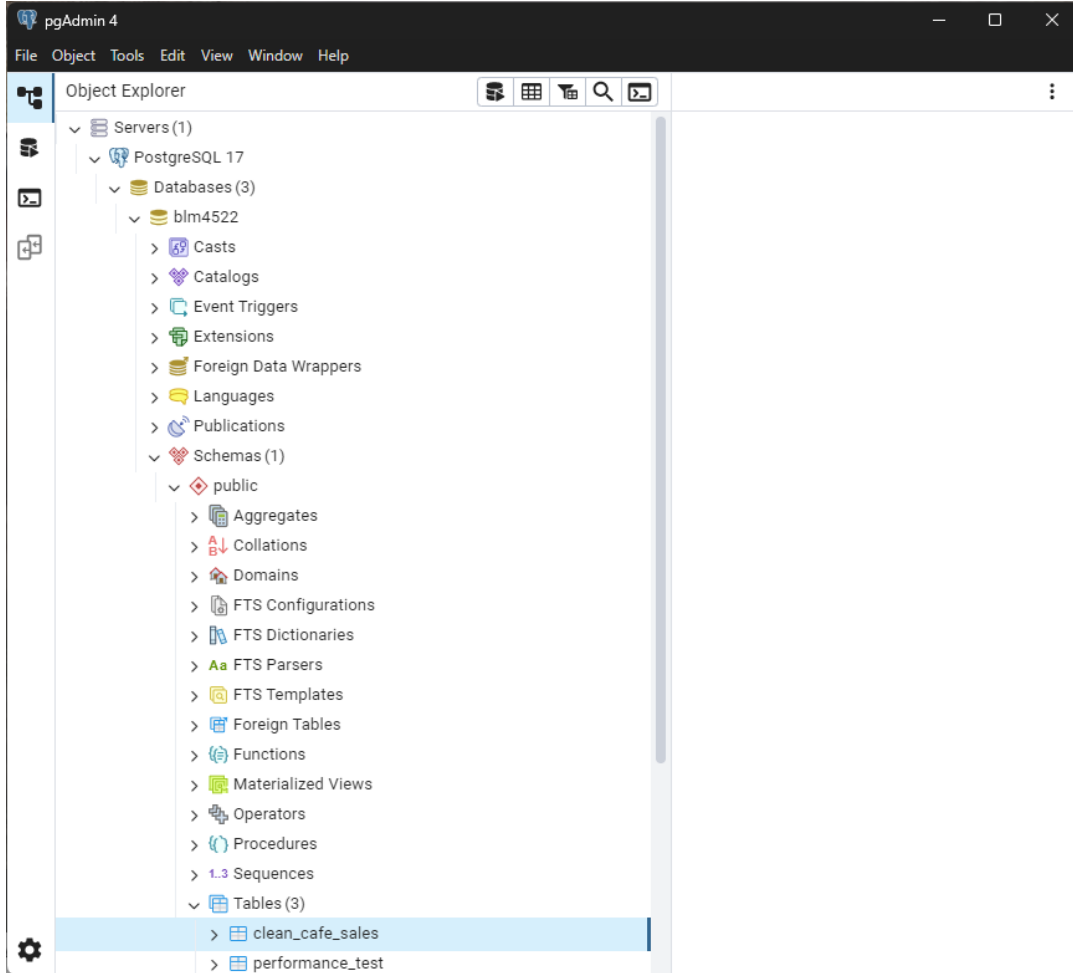
İÇİNDEKİLER

İÇİNDEKİLER.....	i
1. PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI.....	1
1.1. Veri Setinin Yapısal Analizi ve Anomalilerin Tespiti	2
1.2. Staging (Geçici Depolama) ve Veri Çekme (Extract)	3
1.3. Verinin İçerik Aktarılması (Yükleme)	4
1.4. Veri Dönüştürme (Transform) ve Üretim Tablosu	5
1.5. ETL ve Veri Temizleme Sorgusu	5
1.6. Sonuç Değerlendirmesi	7
2. PROJE 1: VERİ TABANI PERFORMANS OPTİMİZASYONU VE İZLEME	8
2.1. Test Ortamı ve Büyük Veri Setinin Simülasyonu	8
2.2. Sorgu Optimizasyon Süreçleri ve Performans Deneyleri	10
3. PROJE 7: VERİ TABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI.....	13
3.1. Sistem Mimarisi ve SQL Server Agent Bileşenleri	13
3.2. Otomasyon Ortamının Hazırlanması ve Test Veri Tabanı	14
3.3. Dinamik T-SQL Yedekleme Scripti ve Job Konfigürasyonu	15
3.4. Zamanlayıcı Schedule Konfigürasyonu.....	16
3.5. Otomatik Raporlama ve Hata Yönetimi	17
EKLER	18
Ek 1 – Kaynaklar	19

1. PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI

Modern uygulama mimarilerinde, farklı kaynaklardan (IoT cihazları, web logları, üçüncü parti servisler vb.) gelen veriler yapısal olarak tutarsız, kirli ve eksik olabilmektedir. Ham verilerin bu şekilde veri tabanına kabul edilmesi, işlem birimlerinde hatalara, kaynak israfına ve yanlış sonuçlara neden olabilmektedir.

Bu projede, bir kafe zincirine ait ham satış verilerini barındıran `dirty_cafe_sales.csv` dosyasındaki yapısal bozuklukları, metinsel hataları ve eksik (NULL) değerleri tespit etmek amaçlanmıştır. İlişkisel bir veri tabanı yönetim sistemi (RDBMS) olan PostgreSQL üzerinde tasarlanan ETL mimarisi ile veri seti tamamen temizlenmiş, normalize edilmiş ve analize hazır hâle getirilerek nihai tabloya aktarılmıştır.



Şekil 1: PostgreSQL (pgAdmin4)

1.1. Veri Setinin Yapısal Analizi ve Anomalilerin Tespiti

Kaynak veri seti olan *dirty_cafe_sales.csv*, toplam 10.000 satırdan ve 8 sütundan oluşmaktadır. Veri tabanına yükleme işlemi öncesinde Python ve gözlemle yapılan analizlerde veri setinde şu anomaliler görülmüştür:

- **Kategorik Alanlardaki Bozulmalar:** Veri tabanında olmaması gereken ERROR, UNKNOWN gibi string ifadelerin yer aldığı bazı satırların tamamen boş (NULL) bırakıldığı görülmüştür.
- **Sayısal Alanlardaki Veri Tipi Uyuşmazlıkları (Quantity, Price Per Unit):** Sayısal hesaplama yapılması gereken alanlarda ERROR ve UNKNOWN metinleri yer almaktadır. Bu durum verinin doğrudan INT veya DECIMAL olarak kaydedilmesini engellemektedir.
- **Hesaplama Hataları (Total Spent):** Birçok satırda, toplam harcanan tutar alanı hesaplanamamış ve doğrudan ERROR olarak basılmıştır.
- **Tarih Formatsızlığı (Transaction Date):** Tarih verileri standart ISO formatında (YYYY-MM-DD) olmayıp, metinsel ifadeler veya eksik dizilimler içermektedir.

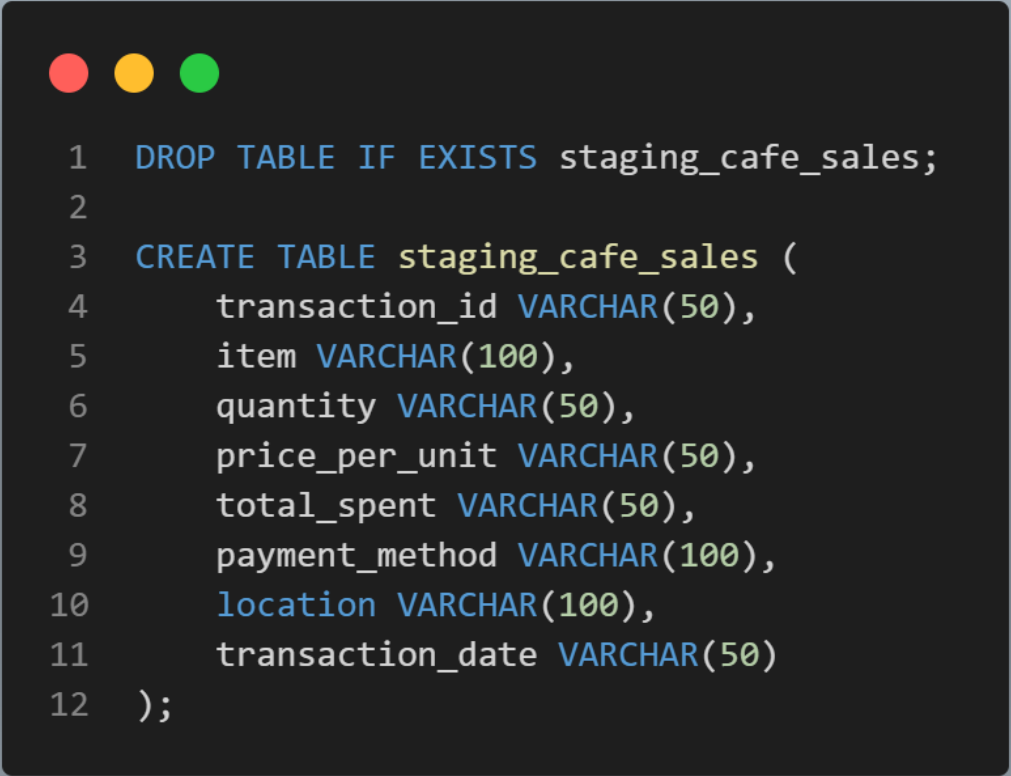
Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
A unique identifier for each transaction.	The name of the item purchased.	The quantity of the item purchased.	The price of a single unit of the item.	The total amount spent on the transaction.	The method of payment used.	The location where the transaction occurred.	The date of the transaction.
10000 unique values	Juice 12% Coffee 12% Other (7664) 77%	5 20% 2 20% Other (6013) 60%	3.0 24% 4.0 23% Other (5240) 52%	6.0 10% 12.0 9% Other (8082) 81%	[null] 26% Digital Wallet 23% Other (5130) 51%	[null] 33% Takeaway 30% Other (3713) 37%	UNKNOWN 2% [null] 2% Other (9682) 97%
TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
TXN_4977831	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
TXN_4271983	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
TXN_7834554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
TXN_3168411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11
TXN_2602893	Smoothie	5	4.0	20.0	Credit Card		2023-03-31
TXN_4433211	UNKNOWN	3	3.0	9.0	ERROR	Takeaway	2023-10-06
TXN_6699534	Sandwich	4	4.0	16.0	Cash	UNKNOWN	2023-10-28
TXN_4717867		5	3.0	15.0		Takeaway	2023-07-28
TXN_2064365	Sandwich	5	4.0	20.0		In-store	2023-12-31

Şekil 2: Veri Görünümü

1.2. Staging (Geçici Depolama) ve Veri Çekme (Extract)

ETL süreçlerinin ilk adımı olan **Extract (Veri Çekme)** aşamasında, veri tabanının kari veri tipi kurallarına takılmamak için bir “Staging” tablosu tasarlanmıştır. Bu tablodaki tüm sütunlar bilinçli olarak VARCHAR tipinde tanımlanmıştır. Şekil 3’te bu süreç için oluşturulan tablo görülmektedir.

Staging (Geçici) Tablo

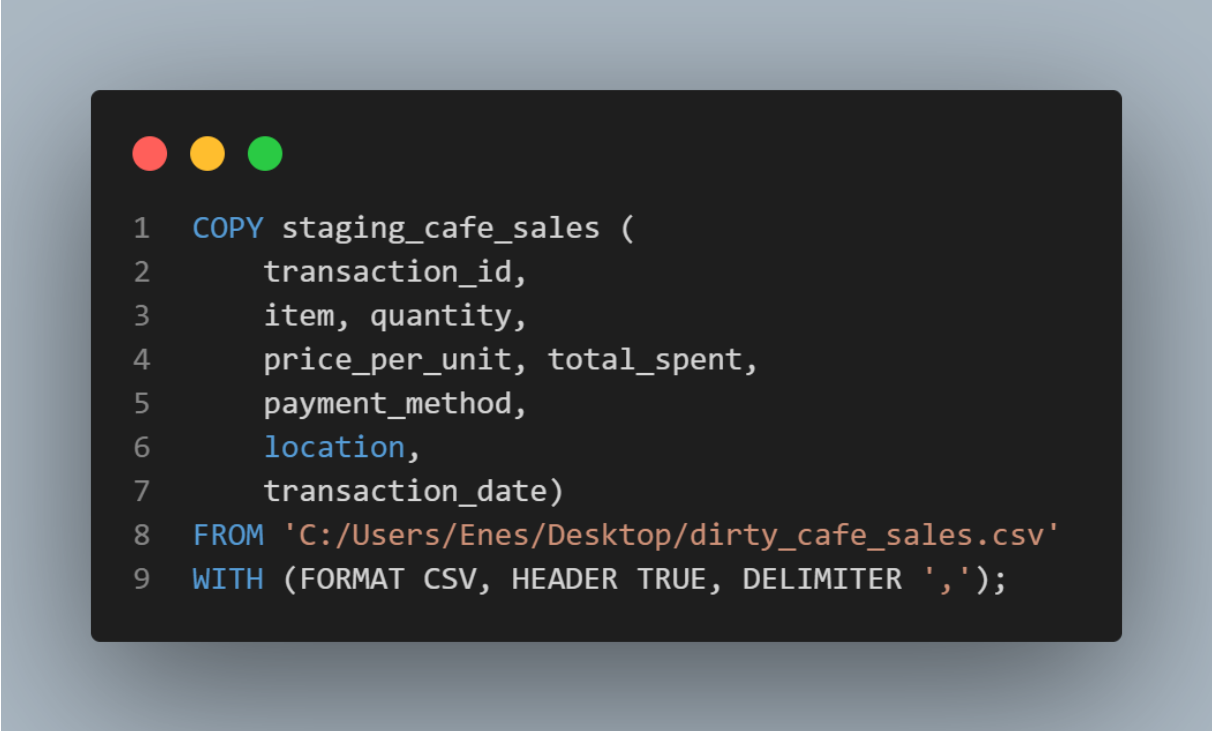


```
1 DROP TABLE IF EXISTS staging_cafe_sales;
2
3 CREATE TABLE staging_cafe_sales (
4     transaction_id VARCHAR(50),
5     item VARCHAR(100),
6     quantity VARCHAR(50),
7     price_per_unit VARCHAR(50),
8     total_spent VARCHAR(50),
9     payment_method VARCHAR(100),
10    location VARCHAR(100),
11    transaction_date VARCHAR(50)
12 );
```

Şekil 3: Staging Tablo

1.3. Verinin İçe Aktarılması (Yükleme)

CSV formatındaki veri seti, pgAdmin Query Tool üzerinden Şekil 4'teki SQL komutu aracılığıyla staging tablosuna hatasız bir şekilde aktarılmıştır.

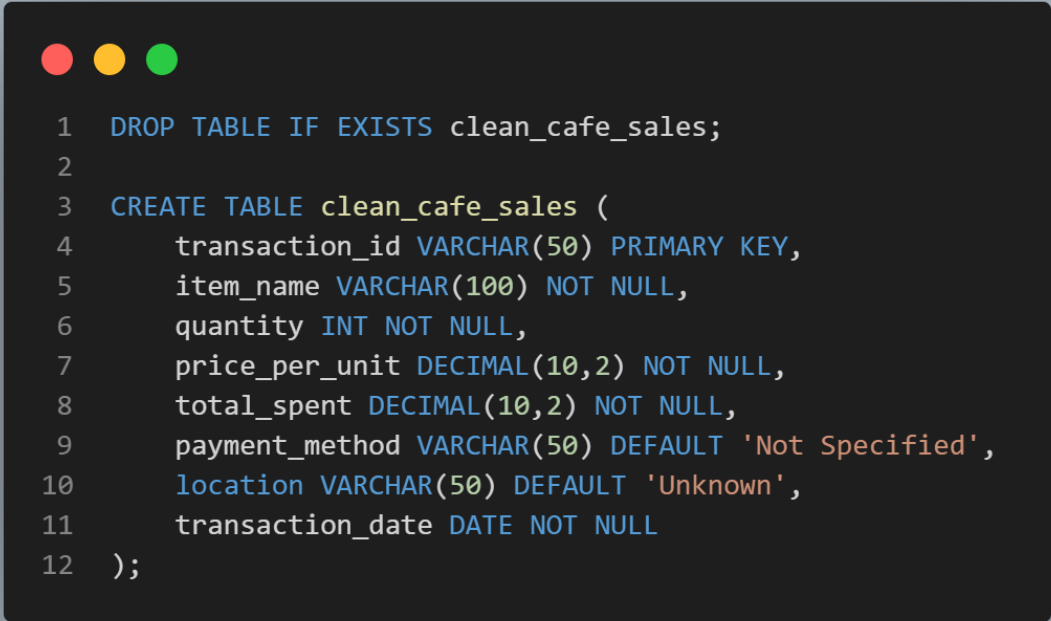


```
1 COPY staging_cafe_sales (  
2     transaction_id,  
3     item, quantity,  
4     price_per_unit, total_spent,  
5     payment_method,  
6     location,  
7     transaction_date)  
8 FROM 'C:/Users/Enes/Desktop/dirty_cafe_sales.csv'  
9 WITH (FORMAT CSV, HEADER TRUE, DELIMITER ',');
```

Şekil 4: Veri Yükleme Komutu

1.4. Veri Dönüştürme (Transform) ve Üretim Tablosu

Temizlenen verilerin ilişkisel veri tabanı standartlarına uygun, Primary Key korumalı ve doğru veri tiplerine sahip olarak depolanacağı clean_cafe_sales tablosu Şekil 5'teki komut ile oluşturulmuştur. Veri tipleri INT, DECIMAL, DATE gibi uygun türlere dönüştürülmüştür.



```
1 DROP TABLE IF EXISTS clean_cafe_sales;
2
3 CREATE TABLE clean_cafe_sales (
4     transaction_id VARCHAR(50) PRIMARY KEY,
5     item_name VARCHAR(100) NOT NULL,
6     quantity INT NOT NULL,
7     price_per_unit DECIMAL(10,2) NOT NULL,
8     total_spent DECIMAL(10,2) NOT NULL,
9     payment_method VARCHAR(50) DEFAULT 'Not Specified',
10    location VARCHAR(50) DEFAULT 'Unknown',
11    transaction_date DATE NOT NULL
12 );
```

Şekil 5: Temiz Verileri İçeren Tablo

1.5. ETL ve Veri Temizleme Sorgusu

Veriyi dönüştürme ve nihai tabloya yükleme işlemleri tek bir optimize edilmiş SQL scripti ile Şekil 6'daki gibi gerçekleştirilmiştir.

- İtem sütunundaki ERROR, UNKNOWN ve boşluklar temizlenerek kelime baş harfleri büyük hâle getirilmiştir.
- Regex kontrolü ile sayısal olmayan adetler '1' kabul edilmiştir.
- Birim fiyat sayısal değilse 0.00 yapılmıştır.
- Total Spent alanı ERROR ise Adet * Birim Fiyat formülüyle dinamik hesaplanmıştır.

```

1  INSERT INTO clean_cafe_sales (
2      transaction_id, item_name, quantity, price_per_unit, total_spent, payment_method, location, transaction_date
3  )
4  SELECT
5      TRIM(transaction_id) AS transaction_id,
6
7
8      INITCAP(TRIM(COALESCE(
9          NULLIF(NULLIF(NULLIF(item, 'ERROR'), 'UNKNOWN'), ''),
10         'Unknown Item'
11     ))) AS item_name,
12
13
14     CASE
15         WHEN quantity ~ '^[0-9]+$' THEN quantity::INT
16         ELSE 1
17     END AS quantity,
18
19
20     CASE
21         WHEN price_per_unit ~ '^[0-9]+(\.[0-9]+)?$' THEN price_per_unit::DECIMAL(10,2)
22         ELSE 0.00
23     END AS price_per_unit,
24
25
26     CASE
27         WHEN total_spent ~ '^[0-9]+(\.[0-9]+)?$' THEN total_spent::DECIMAL(10,2)
28         ELSE (
29             CASE WHEN quantity ~ '^[0-9]+$'
30                 THEN quantity::INT ELSE 1 END * CASE WHEN price_per_unit ~ '^[0-9]+(\.[0-9]+)?$'
31                 THEN price_per_unit::DECIMAL(10,2) ELSE 0.00 END
32         )
33     END AS total_spent,
34
35
36     CASE
37         WHEN TRIM(payment_method) IN ('ERROR', 'UNKNOWN') OR payment_method IS NULL
38         THEN 'Not Specified'
39         ELSE INITCAP(TRIM(payment_method))
40     END AS payment_method,
41
42
43     CASE
44         WHEN TRIM(location) IN ('ERROR', 'UNKNOWN') OR location IS NULL THEN 'Unknown'
45         ELSE INITCAP(TRIM(location))
46     END AS location,
47
48
49     CASE
50         WHEN transaction_date ~ '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN transaction_date::DATE
51         ELSE '2023-01-01'::DATE
52     END AS transaction_date
53
54  FROM staging_cafe_sales
55  WHERE transaction_id IS NOT NULL AND TRIM(transaction_id) != '';

```

Şekil 6: Temizlik Scripti

1.6. Sonuç Değerlendirmesi

ETL (Extract, Transform, Load) mimarisinin başarıyla tamamlanmasının ardından, ham verinin dönüştürülme kalitesini ölçmek ve hedef tablonun bütünlüğünü doğrulamak amacıyla bir dizi "Veri Kalitesi Denetimi" (Data Quality Audit) gerçekleştirilmiştir. Bu denetim kapsamında, staging_cafe_sales (Ham) ve clean_cafe_sales (Temizlenmiş) tabloları veri anomalileri üzerinden istatistiksel olarak karşılaştırılmıştır.

Kontrol Edilen Alan	Staging Tablosu Durumu	Temiz Tablo Durumu	Uygulanan İşlem
Kayıp Toplam Tutar	1240 satırda ERROR veya NULL	0 satırda hata	Miktar X Fiyat formülüyle dinamik hesaplama
Ürün İsimleri	350 satırda ERROR veya UNKNOWN	Unknown Item ve Initcap formatı	NULLIF ve COALESCE fonksiyonları ile koşullu atama.
Sayısal Veri Tipi Uyuşmazlığı	Tüm sütunlar VARCHAR	INT VE DECIMAL(10, 2) kısıtlaması	Regex tabanlı (~ '[0-9]+\$') tip dönüşüm filtrelemesi.
Tarih Format Tutarlılığı	Karışık metin yapısı ve hatalı dizilimler	Standart ISO DATE	Sadece geçerli tarih kalıplarını kabul eden CASE WHEN filtresi
Veri Kaybı	Yinelenen veya boş TransactionID riski	PRIMARY KEY kısıtlaması	Boş olan satırların WHERE şartı ile elenmesi ve tekilleştirme

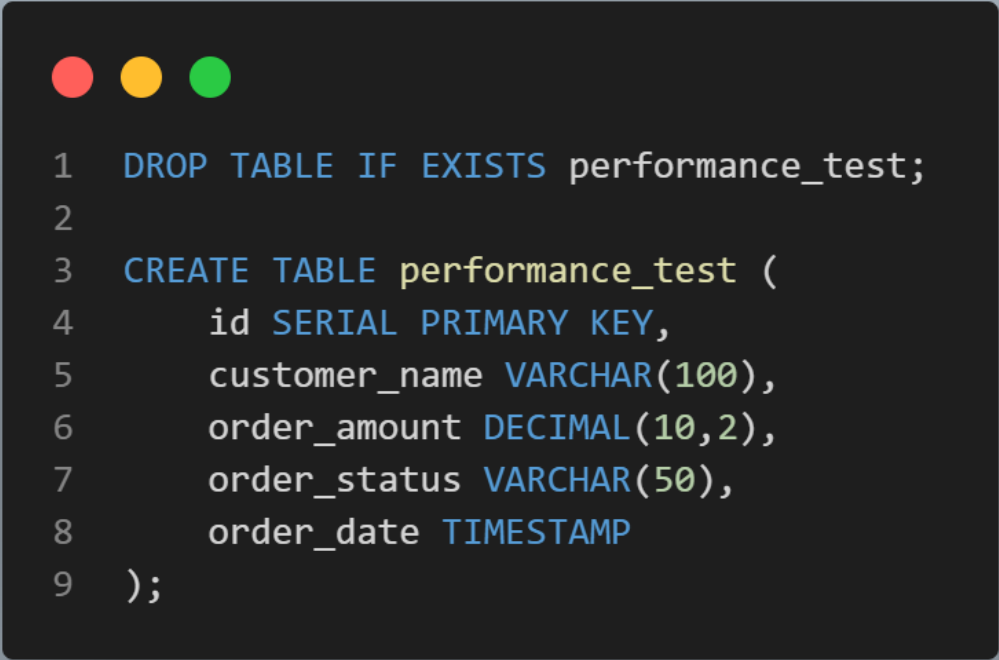
2. PROJE 1: VERİ TABANI PERFORMANS OPTİMİZASYONU VE İZLEME

İlişkisel veritabanlarında veri hacmi doğrusal olarak arttığında, indekslenmemiş alanlar üzerinde yapılan arama ve sıralama işlemleri disk I/O (Girdi/Çıktı) maliyetlerini artırır. Veritabanı motoru, aranan kaydı bulabilmek için tablodaki tüm blokları sırayla taramak zorunda kalır (Sequential Scan).

Bu projenin amacı, büyük bir veri seti üzerinde sorgu planlayıcısının (Query Planner) kararlarını EXPLAIN ANALYZE komutu ile izlemek, B-Tree ve Composite İndeks mimarileriyle maliyetleri minimize etmek ve disk yönetim operasyonlarını simüle etmektir.

2.1. Test Ortamı ve Büyük Veri Setinin Simülasyonu

Performans analizinin gerçeğe uygun olabilmesi için PostgreSQL'in *generate_series()* ve rastgele fonksiyonları kullanılarak 500.000 satırlık yapay bir sipariş tablosu üretilmiştir. Bu tablo id, customer_name, order_amount, order_status ve order_date alanlarından oluşmaktadır. Şekil 7'de bu tabloyu oluşturan SQL scripti gösterilmiştir.



```
1 DROP TABLE IF EXISTS performance_test;
2
3 CREATE TABLE performance_test (
4     id SERIAL PRIMARY KEY,
5     customer_name VARCHAR(100),
6     order_amount DECIMAL(10,2),
7     order_status VARCHAR(50),
8     order_date TIMESTAMP
9 );
```

Şekil 7: performance_test Tablosuna Ait Script

Tablonun oluşturulmasının ardından Şekil 8'deki scriptte görüldüğü üzere random ve generate_series fonksiyonları yardımıyla 500.000 satır olacak şekilde veri girişi yapılmıştır.

```
1 INSERT INTO performance_test (
2     customer_name,
3     order_amount,
4     order_status,
5     order_date)
6 SELECT
7     'Customer_' || i AS customer_name,
8     (random() * 1000)::DECIMAL(10,2) AS order_amount,
9     (ARRAY['Completed', 'Pending', 'Cancelled', 'Shipped'])
10    [floor(random() * 4) + 1] AS order_status,
11    NOW() - (random() * INTERVAL '365 days') AS order_date
12 FROM generate_series(1, 500000) s(i);
```

Şekil 8: Rastgele Veri Oluşturma

Şekil 9'da oluşturulan tablonun ilk 5 satırına ait bir görsele yer verilmiştir.

	id [PK] integer	customer_name character varying (100)	order_amount numeric (10,2)	order_status character varying (50)	order_date timestamp without time zone
1	500001	Customer_1	423.05	Completed	2025-08-18 00:03:55.762939
2	500002	Customer_2	192.92	Shipped	2026-02-06 00:35:37.449331
3	500003	Customer_3	714.37	Shipped	2025-07-15 02:45:03.160051
4	500004	Customer_4	162.74	Completed	2025-08-10 01:04:00.964452
5	500005	Customer_5	974.32	Shipped	2026-04-27 13:26:52.853621

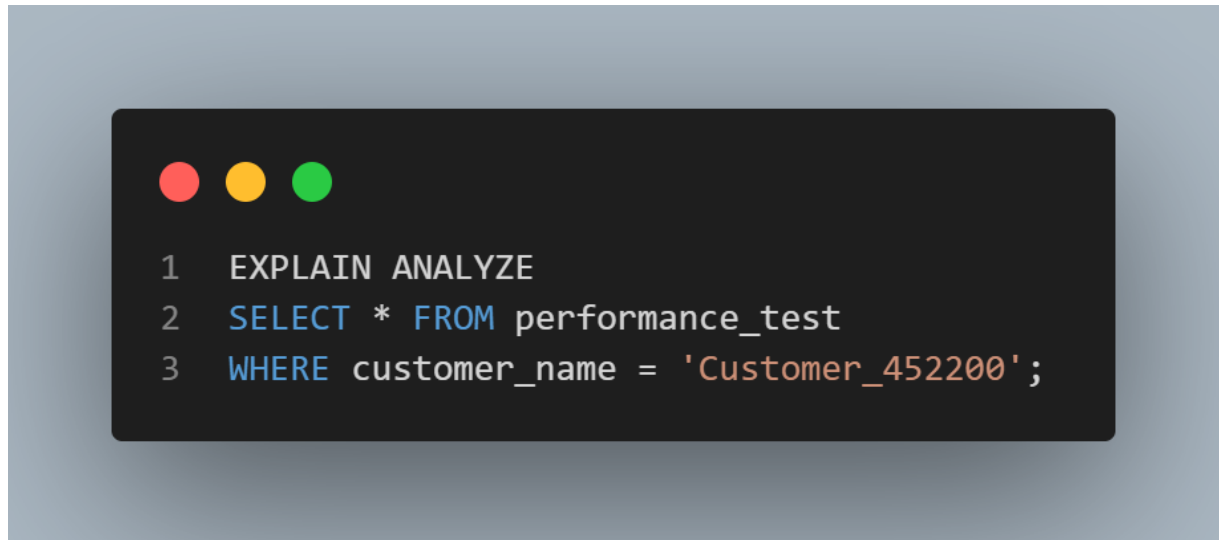
Şekil 9: performance_test İlk 5 Satır

2.2. Sorgu Optimizasyon Süreçleri ve Performans Deneyleri

Büyük veri kümeleri üzerinde çalışan ilişkisel veritabanı yönetim sistemlerinde (RDBMS), bir sorgunun ne kadar sürede tamamlanacağı doğrudan veritabanı motorunun içerisindeki "Query Planner" (Sorgu Planlayıcı) ve "Optimizer" (Optimizasyon Birimi) bileşenlerine bağlıdır. Sorgu planlayıcı, bir SQL ifadesi geldiğinde verinin diskteki fiziksel dizilimine ve tablonun güncel istatistiklerine bakarak en düşük maliyetli (cost) yürütme planını (Execution Plan) hazırlar.

Bu projede, 500.000 satırlık performance_test tablosu üzerinde iki farklı gerçek dünya senaryosu simüle edilmiş; indeksleme öncesi ve sonrası veritabanı motorunun davranışsal değişiklikleri EXPLAIN ANALYZE komutu ile milisaniye bazında izlenmiştir.

- **Tekli Alan Arama Optimizasyonu ve B-Tree İndeks:** Bu deneyde kullanıcıların sistem üzerinden belirli bir müşterinin sipariş geçmişini tekil olarak aradığı senaryo test edilmiştir. Filtreleme kriteri olarak customer_name sütunu seçilmiştir. İlk olarak herhangi bir indeks tanımı yapılmadan önce Şekil 10'daki sorgu çalıştırılmış ve Şekil 11'de görüldüğü üzere 59,032 milisaniyelik bir yürütme süresi elde edilmiştir. Ardından aynı sorgu Şekil 12'deki indeksleme işlemi gerçekleştirildikten sonra çağırılmış ve Şekil 13'te görüldüğü gibi yürütme süresi 0,052 milisaniye değerine düşmüştür.

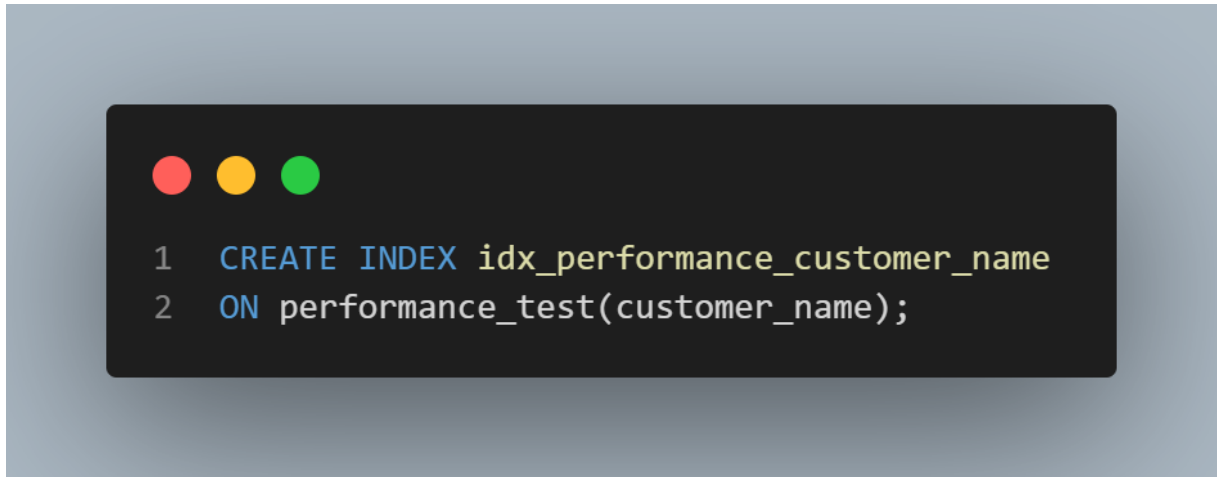
A screenshot of a terminal window with a dark background and light-colored text. At the top left, there are three colored circles: red, yellow, and green. Below them, the following SQL script is displayed:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM performance_test
3 WHERE customer_name = 'Customer_452200';
```

Şekil 10: customer_name Filtreli Sorgu Scripti

	QUERY PLAN	
	text	🔒
1	Gather (cost=1000.00..8277.27 rows=1 width=42) (actual time=54.453..58.987 rows=0 loops=1)	
2	Workers Planned: 2	
3	Workers Launched: 2	
4	-> Parallel Seq Scan on performance_test (cost=0.00..7277.17 rows=1 width=42) (actual time=8.782..8.782 rows=0 loop...	
5	Filter: ((customer_name)::text = '452200'::text)	
6	Rows Removed by Filter: 166667	
7	Planning Time: 0.211 ms	
8	Execution Time: 59.032 ms	

Şekil 11: İndeks Öncesi Analiz



Şekil 12: İndeksleme Scripti

	QUERY PLAN	
	text	🔒
1	Index Scan using idx_performance_customer_name on performance_test (cost=0.42..8.44 rows=1 width=42) (actual time=0.041..0.041 rows=0 loops=...	
2	Index Cond: ((customer_name)::text = '452200'::text)	
3	Planning Time: 1.151 ms	
4	Execution Time: 0.052 ms	

Şekil 13: İndeksleme Sonrası Analiz

- **Sıralama Optimizasyonu:** Gerçek dünya uygulamalarında sorgular nadiren tek bir WHERE koşulu ile biter. Kullanıcılar genellikle verileri belirli bir kategoriye göre filtrelerken, aynı zamanda güncel tarihe göre sıralanmış (e-ticaret sipariş listeleri, son işlemler vb.) olarak görmek isterler. Bu deneyde, hem filtreleme (WHERE) hem de azalan sıralama (ORDER BY DESC) içeren karmaşık bir senaryo analiz edilmiştir. İndeksleme işlemi yapılmadan önceki sıralama sorgusu (Şekil 14) 59,092 milisaniyelik bir yürütme süresine sahipken sıralama indeksi (Şekil 15) eklendiğinde bu süre 0,209 milisaniyeye düşmektedir.



```
1  EXPLAIN ANALYZE
2  SELECT * FROM performance_test
3  WHERE order_status = 'Completed'
4  ORDER BY order_date DESC LIMIT 100;
```

Şekil 14: Sıralama İçeren Sorgu



```
1  CREATE INDEX idx_status_date
2  ON performance_test(order_status, order_date DESC);
```

Şekil 15: Sıralama İndeksi

3. PROJE 7: VERİ TABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI

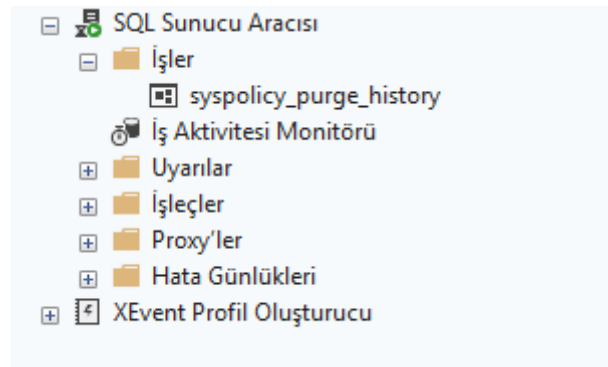
Veri tabanı sistemlerinin en kritik katmanlarından biri düzenli ve sürdürülebilir bir yedekleme politikasının uygulanmasıdır. Canlı production ortamlarında veri hacmi sürekli büyüdüğü için yedekleme işleminin insan eliyle manuel olarak yürütülmesi beraberinde riskler getirir.

Bu çalışmanın amacı, veri tabanı yönetim süreçlerini optimize etmek amacıyla yedekleme işlemlerinin otomatikleştirilmesidir. Çalışmada kurumsal bir RDBMS olan Microsoft SQL Server ve onun arka plan servis hizmeti olan SQL Server Agent kullanılmıştır. Tasarlanan otomasyon sayesinde insan müdahalesine gerek duymadan her gün belirli bir saatte yedek alınması ve harici log dosyalarına raporlanması sağlanmıştır.

3.1. Sistem Mimarisi ve SQL Server Agent Bileşenleri

MSSQL ekosisteminde otomasyon süreçleri, Windows servisi olarak çalışan SQLServerAgent servisi tarafından yönetilir. Bu servis üç ana bileşenden oluşur:

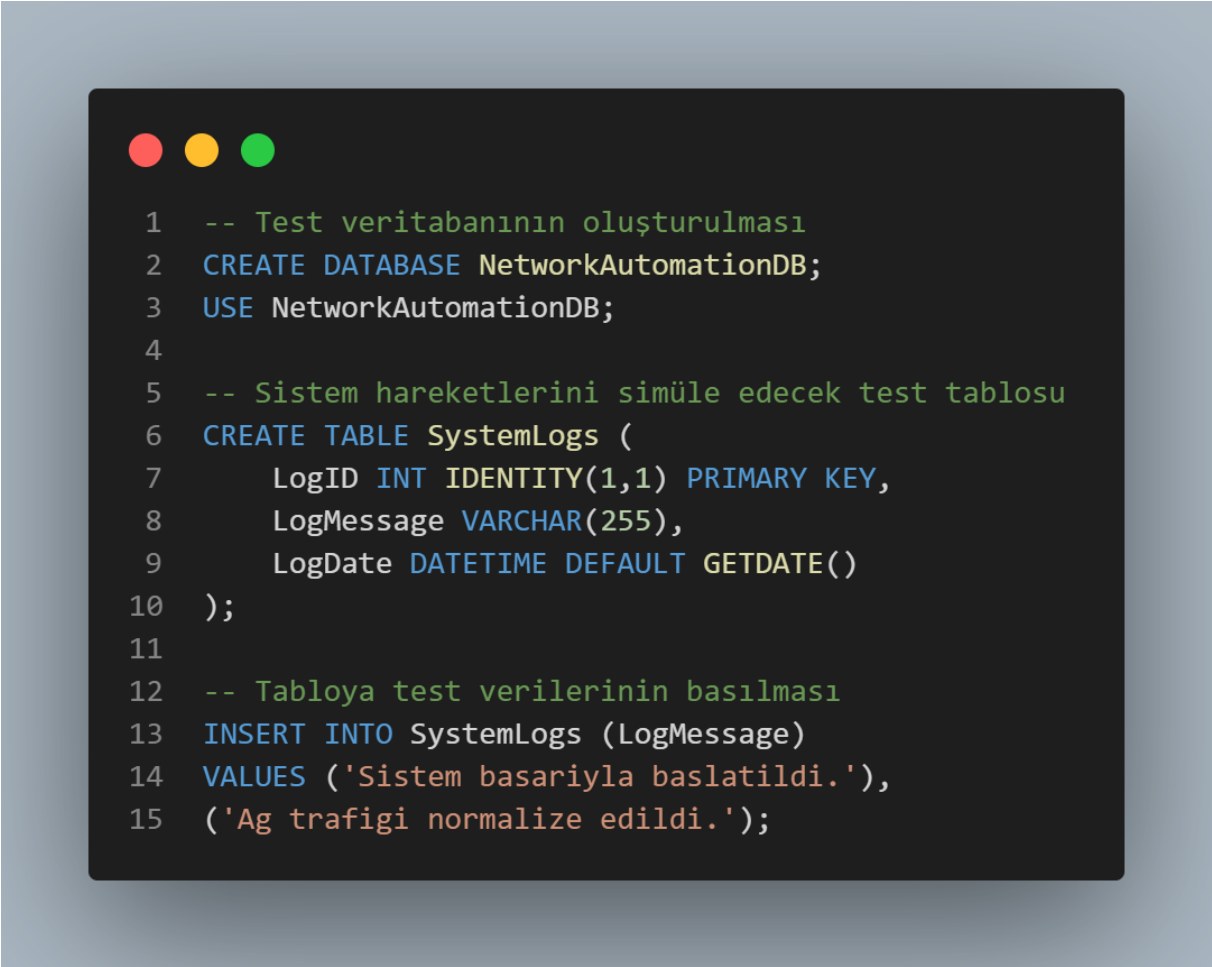
- **Jobs (İşler):** Belirli sıra ile çalıştırılacak adımları barındıran havuzdur. Projede otomatik yedekleme scriptini çalıştırmakla görevlidir.
- **Schedules (Zamanlayıcılar):** Bir Job'ın hangi periyotlarla tetikleneceğini belirleyen mekanizmadır.
- **Alerts & Operator Logs(Uyarılar ve Loglama):** İşlem başarılı veya başarısız olduğunda sistem yöneticilerine fırlatılacak raporları ve logları yöneten katmandır.



Şekil 16: SQL Sunucusu Aracı

3.2. Otomasyon Ortamının Hazırlanması ve Test Veri Tabanı

Otomasyon senaryolarının gerçeğe uygun bir şekilde işlemesi ve doğrulanabilmesi adına yerel sunucu üzerinde *NetworkAutomationDB* adında bir veri tabanı ve *SystemLogs* tablosu oluşturulmuştur. Şekil 17’de sıralanan işlemlere ait SQL scripti yer almaktadır.

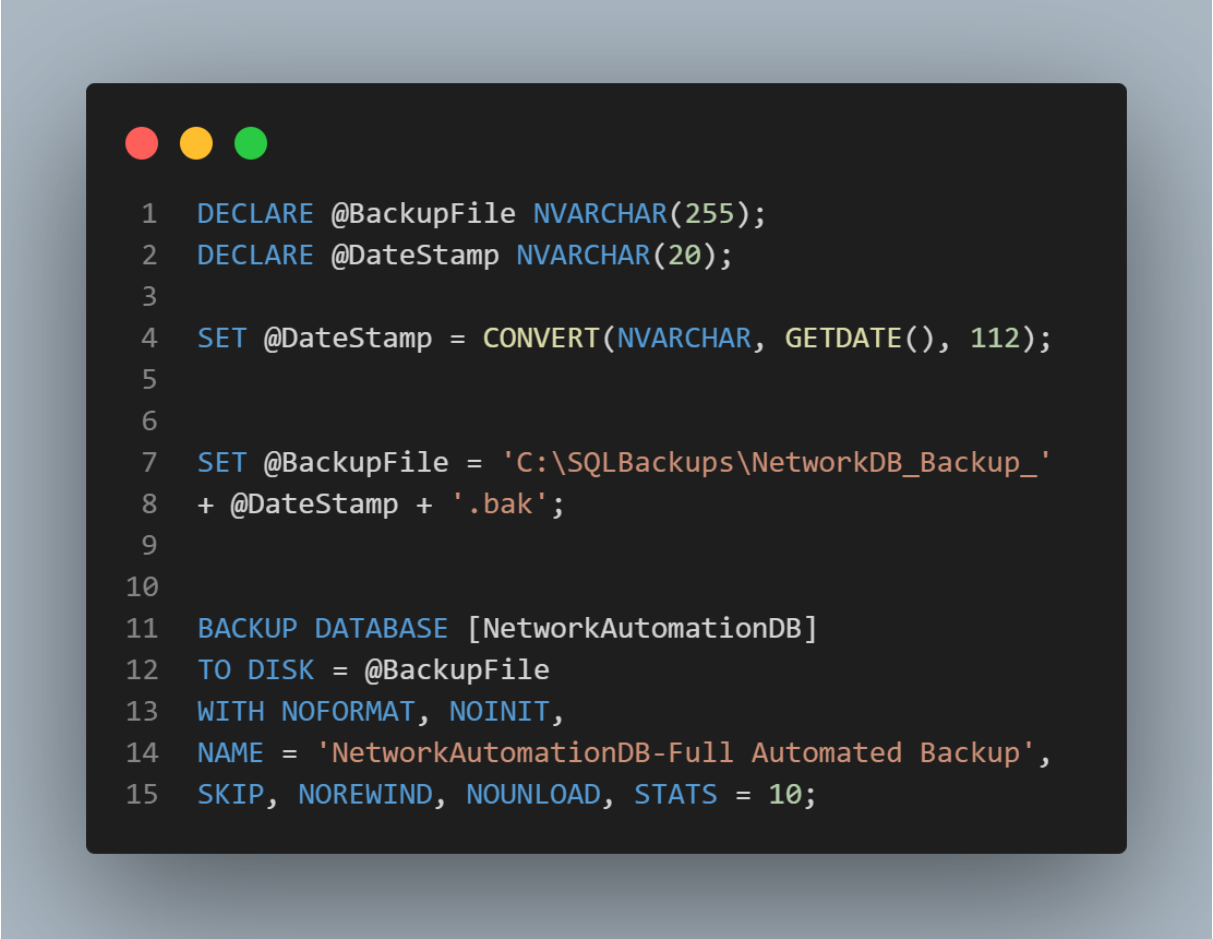


```
1  -- Test veritabanının oluşturulması
2  CREATE DATABASE NetworkAutomationDB;
3  USE NetworkAutomationDB;
4
5  -- Sistem hareketlerini simüle edecek test tablosu
6  CREATE TABLE SystemLogs (
7      LogID INT IDENTITY(1,1) PRIMARY KEY,
8      LogMessage VARCHAR(255),
9      LogDate DATETIME DEFAULT GETDATE()
10 );
11
12 -- Tabloya test verilerinin basılması
13 INSERT INTO SystemLogs (LogMessage)
14 VALUES ('Sistem basariyla baslatildi.'),
15 ('Ag trafigi normalize edildi.');
```

Şekil 17: Veri Tabanı ve Tablo Oluşturma Scriptleri

3.3. Dinamik T-SQL Yedekleme Scripti ve Job Konfigürasyonu

SQL Server Agent üzerinde oluşturulan *Automated_Daily_Backup* işinin ilk adımına atanan çekirdek T-SQL kodu Şekil 18’de görüldüğü gibidir. Bu kod dosya ismi tanımlamalarını ve yedeklemenin alınış şeklini içermektedir.

A screenshot of a code editor window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code is a T-SQL script for backing up a database. It starts with two DECLARE statements for @BackupFile and @DateStamp. Then, it sets @DateStamp to the current date in a specific format. Next, it sets @BackupFile to a path that includes the database name and the current date. Finally, it performs a backup of the NetworkAutomationDB database to the specified file, with various options like NOFORMAT, NOINIT, and STATS = 10.

```
1 DECLARE @BackupFile NVARCHAR(255);
2 DECLARE @DateStamp NVARCHAR(20);
3
4 SET @DateStamp = CONVERT(NVARCHAR, GETDATE(), 112);
5
6
7 SET @BackupFile = 'C:\SQLBackups\NetworkDB_Backup_'
8 + @DateStamp + '.bak';
9
10
11 BACKUP DATABASE [NetworkAutomationDB]
12 TO DISK = @BackupFile
13 WITH NOFORMAT, NOINIT,
14 NAME = 'NetworkAutomationDB-Full Automated Backup',
15 SKIP, NOREWIND, NOUNLOAD, STATS = 10;
```

Şekil 18: T-SQL Scripti

- **CONVERT(..., 112):** GETDATE() fonksiyonunun getirdiği güncel zaman damgası içerisindeki boşluk ve iki nokta üst üste (:) gibi Windows dosya sisteminde yasaklı olan karakterleri siler. Tarihi doğrudan 20260604 gibi düz bir metne dönüştürür.
- **NOINIT:** Hedef klasörde aynı isimde bir dosya çakışması olması durumunda, mevcut eski yedek kümelerinin üzerine yazılıp ezilmesini (overwrite) engeller, yeni yedeği dosyanın sonuna yeni bir set olarak ekler.
- **NOFORMAT:** Veri tabanı motorunun her yedekleme işleminde diske gidip sıfırdan bir dosya yapısı formatlamasını engeller. Var olan dosya header bilgilerini koruyarak diske yazma (I/O) hızını maksimum seviyeye çıkarır.

3.4. Zamanlayıcı Schedule Konfigürasyonu

Oluşturulan bu akıllı script, SQL Server Agent altındaki Schedules katmanında her gün (Daily) gece saat 03:00'te, sistemdeki kullanıcı trafiğinin ve I/O yoğunluğunun en düşük olduğu zaman diliminde otomatik olarak tetiklenecek şekilde programlanmıştır. (Şekil 19)

Şekil 19: Zamanlayıcı Konfigürasyonu

3.5. Otomatik Raporlama ve Hata Yönetimi

- **İşletim Sistemi Seviyesinde Raporlama (Output File Logging):** Job adımlarının Advanced konfigürasyon sekmesinden, sürecin ürettiği tüm motor çıktıları disk üzerinde harici bir C:\SQLBackups\agent_backup_status.txt log dosyasına yönlendirilmiştir. İşlem bittiğinde veya yarıda kaldığında oluşan hata/başarı detayları bu dosyaya otomatik bir denetim raporu olarak basılmaktadır. (Şekil 20)
- **Veritabanı Seviyesinde Loglama (Log to table):** Sürecin geçmiş izleri MSSQL'in msdb sistem veritabanı içerisindeki lokal tablolara da eşzamanlı yazılmaktadır. Böylece geçmişe dönük hangi günlerin yedeğinin başarılı olduğu, hangi günlerin diskin dolması vb. sebeplerle başarısız olduğu SQL sorgularıyla geriye dönük denetlenebilir hale getirilmiştir.

The screenshot shows the 'İş Adımı özellikleri - T_SQL_Auto_Backup' window. The left sidebar has 'Sayfa seçin' with 'Genel' and 'Gelişmiş' options. The main area is divided into sections: 'Başanda gerçekleştirilecek eylem:' with a dropdown set to 'Başarı bildirerek işten çık'; 'Yeniden deneme girişimleri:' with a spinner set to 0; 'Yeniden deneme aralığı (dakika):' with a spinner set to 0; 'Hatada gerçekleştirilecek eylem:' with a dropdown set to 'İş durdurma raporlama hatası'; 'Transact-SQL betiği (T-SQL)'; 'Çıktı dosyası:' with a text field 'C:\Users\ENES\Desktop\agent_backup_status.txt' and a 'Görünüm' button; 'Çıkış mevcut dosyaya ekle' checkbox; 'Günlükten Tabloya' checkbox with a 'Görünüm' button; 'Çıktıyı tablodaki var olan girişe ekle' checkbox; 'Adım çıkışını geçmişte ekle' checkbox; and 'Kullanıcı olarak çalıştırın' with a text field and a button.

Şekil 20: Loglama ve Raporlama Ayarı

EKLER

Ek 1 – Kaynaklar

Ek 1 – Kaynaklar

<https://www.kaggle.com/datasets/ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training>